

# Operating Systems Exercise 3

---

Course: Operating Systems - Reichman University

Due date: 27.06.2026 at 23:59 (submit via INGIInious; no late submissions)

## Acknowledgments

---

## Submission rules (read carefully)

---

- Work in **pairs** (exactly two students per team).
- Submission is made through the INGIInious system.
  - For the programming part, submit a single ZIP file containing `fs.c`, and any private helper `.c/.h` files you add.
    - Do **not** modify `fs.h`, including the structs. Also, do **not** submit `fs.h`.
    - Do **not** submit compiled binaries.
    - Do **not** submit `test.c`. You should write it locally to test your implementation, but the submitted implementation files must **not** contain a `main()` function.
  - For the theoretical part, submit a PDF file containing your answers. Hand-written scans in the PDF are **not** approved and will be declined. The submission must be typed, including math.
  - When you submit, also include your names and IDs according to the accepted format. See the instructions in the INGIInious submission page.
- The assignment is graded on Ubuntu **x86\_64** architecture in the INGIInious environment.
  - However, local execution may **not be identical** to the grading environment. Make sure your code also works successfully in INGIInious.
- The INGIInious grader uses **Ubuntu 24.04 LTS**, **GCC 13**, and **GNU/C 17** (C17 with GNU extensions). To compile by yourself, use `gcc-13 -std=gnu17`. Make sure your code works successfully in this environment.
- You may submit your exercise as many times as you want until the due date. Your last submission is the one that will be graded. Submitting after the due date without explicit approval will not be graded and will be considered as not submitted.
- All source code will be scanned using plagiarism and LLM-detection tools. If suspicious similarity is detected, the team(s) may be investigated for academic misconduct according to university policy.

## Part A - Theory (submit as a separate PDF)

---

Submit answers in a single PDF file.

### TQ1 - Scheduling: A Multi-Level Feedback Queue (MLFQ) variant

#### Problem

Consider the following **variant** of a Multi-Level Feedback Queue (MLFQ) scheduler with three priority queues:

| Queue | Quantum |
|-------|---------|
| Q0    | 10 ms   |
| Q1    | 20 ms   |
| Q2    | 40 ms   |

Rules:

- New threads start at Q0 (highest priority).
- A thread that uses its full quantum is demoted one level (a thread already in Q2 stays in Q2).
- A thread that yields or blocks before its quantum expires stays at its current level.
- The scheduler always dispatches from the highest-priority non-empty queue. Within a queue, threads are served in FIFO order.

Note: This is one specific variant of MLFQ; other variants behave differently (for example, *promoting* a thread that yields). Answer according to the rules stated above.

Two threads arrive at  $t = 0$ , each needing 1000 ms of CPU time in total:

- **Thread A** is genuinely CPU-bound: it runs until it is preempted.
- **Thread B** is also CPU-bound, but tries to *masquerade* as an I/O-bound thread: it calls `yield()` after exactly 9.9 ms of every quantum it receives.

### Tasks

1. Once the system reaches a steady state (after the initial scheduling rounds), in which queue is each thread?
2. At most how many full quanta does Thread A complete before Thread B takes over the CPU? In steady state, while both threads are still alive, what percentage of CPU time does Thread A receive? Explain in one sentence.
3. In one sentence: what does this MLFQ variant incorrectly assume about Thread B?
4. Under these rules, can Thread B ever be demoted? Why or why not? (One sentence.)

## TQ2 - Memory: Demand paging and page replacement

### Problem

A process runs on a machine with a page size of 4 KB and **demand paging**: a virtual page is brought into a physical frame only when it is first accessed. Only **3 physical frames** are available to this process.

Each page table entry (PTE) has a **valid bit** (is the page in a frame?), a **reference bit** (used by Clock), and a **dirty bit** (was the page modified since it was loaded?).

The process accesses this *reference string*, from left to right:

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Clock (approximate-LRU) convention — use exactly this:

- The 3 frames form a circle. The first three faults fill Frame 0, Frame 1, Frame 2 in order.
- A page loaded into a frame gets reference bit = 1.
- On a **hit**, set that page's reference bit to 1.

- On a fault when all frames are full, start at the hand. If the page it points to has reference bit **1**, clear it to **0** and advance the hand, repeating until a page with reference bit **0** is found. Evict that page, load the new page in its place (reference bit **1**), and advance the hand one position past it.

In the Clock table, the **Hand** column gives the hand position *after* the access has been processed.

## Tasks

- Fill in the blanks: a page fault occurs when the accessed page's PTE has valid bit = \_\_\_\_ . The CPU then traps to the OS, which loads the page into a \_\_\_\_ .
- Simulate the reference string under **LRU** by completing the table, then give the total number of page faults.

| Access         | 1   | 2 | 3 | 4 | 1 | 2 | 5 | 1 | 2 | 3 | 4 | 5 |
|----------------|-----|---|---|---|---|---|---|---|---|---|---|---|
| Fault? (F/H)   | F   |   |   |   |   |   |   |   |   |   |   |   |
| Resident pages | {1} |   |   |   |   |   |   |   |   |   |   |   |

Total LRU page faults: \_\_\_\_

- Simulate the same reference string under **Clock** (convention above) by completing the table, then give the total number of page faults. Each frame cell is (page, reference bit) .

| Access | F/H | Frame 0 | Frame 1 | Frame 2 | Hand |
|--------|-----|---------|---------|---------|------|
| 1      | F   | (1,1)   | —       | —       | 1    |
| 2      |     |         |         |         | 2    |
| 3      |     |         |         |         | 0    |
| 4      |     |         |         |         |      |
| 1      |     |         |         |         |      |
| 2      |     |         |         |         |      |
| 5      |     |         |         |         |      |
| 1      |     |         |         |         |      |
| 2      |     |         |         |         |      |
| 3      |     |         |         |         |      |
| 4      |     |         |         |         |      |
| 5      |     |         |         |         |      |

Total Clock page faults: \_\_\_\_

- Exact LRU produced \_\_\_\_ faults, Clock produced \_\_\_\_ faults. **True or False:** an approximate policy like Clock can never cause fewer faults than exact LRU. Justify in one sentence.
- When the OS evicts a victim, exactly one case requires an extra write to swap before the frame is reusable: a **clean** page or a **dirty** page? (Pick one.) In one sentence, why?

## TQ3 - File Systems: links, partitions, and indirect blocks

### Problem

Alice's Linux laptop has two separate **ext4** partitions: one mounted at `/home`, the other at `/data`. She has a 5 GiB file at `/home/alice/report.pdf`, plus a **hard link** to the same file at `/home/alice/backup/report.pdf`.

The two commands below are independent alternatives. Each starts from the original state above.

- **Command A:** `mv /home/alice/report.pdf /home/alice/archive/report.pdf`
- **Command B:** `mv /home/alice/report.pdf /data/report.pdf`

### Tasks

1. Command A copies \_\_\_\_ bytes of the file's content; Command B copies \_\_\_\_ bytes. In one sentence, what does Command A actually change on disk?
2. Originally, two hard links point to the file's inode. After **Command B** removes the entry `/home/alice/report.pdf`, how many hard links still point to that `/home` inode: \_\_\_\_? Running `cat /home/alice/backup/report.pdf` then shows (pick one): original content / empty output / error. In one sentence, why?
3. **True or False:** a hard link can point to a file whose inode lives on a different partition. Then, in one sentence, explain why `ln /home/alice/backup/report.pdf /data/hardlink.pdf` fails.
4. Suppose `/home/alice/backup/report.pdf` were a **symbolic link** to `/home/alice/report.pdf` instead of a hard link. After **Command B**, `cat /home/alice/backup/report.pdf` shows (pick one): original content / empty output / error (broken link). In one sentence, why?
5. The inodes on `/data` use the structure taught in class: **12 direct, 1 single-indirect, 1 double-indirect, 1 triple-indirect** block pointers, with **4 KiB** blocks and **4-byte** pointers. A file occupies **5 GiB**. ("Data blocks" = blocks holding file content, not the indirect pointer blocks.)
  1. How many block pointers fit in one indirect block? \_\_\_\_
  2. How many data blocks does the 5 GiB file occupy? \_\_\_\_
  3. How many data blocks can be reached using only the direct + single-indirect + double-indirect pointers? \_\_\_\_
  4. Is the triple-indirect pointer needed for this file? (yes/no)

## Part B - Programming

---

### Introduction

---

In this assignment, you will implement a block-based filesystem in C, contained entirely within a single disk-image file. Your implementation, called "**OnlyFiles**", will be a simplified filesystem that supports file operations without directories. By building your own miniature filesystem — modeled conceptually on something like ext4 — you will gain hands-on experience with the core concepts of: low-level I/O, on-disk data structures, metadata management, and robust error handling.

The filesystem will store its data in a regular file (10 MB in size), which acts as a "virtual disk". You will implement a C API that allows formatting the filesystem, mounting and unmounting it, creating and deleting files, and reading from and writing to files.

# Learning objectives

By completing this assignment, you will:

- Understand how filesystems manage data at the block level.
- Implement core filesystem operations (create, read, write, delete).
- Work with system-level I/O calls instead of buffered I/O.
- Gain practical experience with filesystem metadata structures.
- Practice error handling for filesystem operations.

## Provided files

You are given the following files:

- `fs.h` — the library header file defining all constants, data structures, and function prototypes.
- `test.c` — a simple test.

Do **not** modify `fs.h`, including the structs.

Do **not** submit `fs.h`, `test.c`, as the INGenious checker will use the official versions of these files.  
Do **not** submit any other source file that includes a `main` function.

## Disk layout

Your virtual disk (10 MB) is divided into fixed-size blocks of 4 KB each, resulting in a total of 2560 blocks. These blocks are organized into four main regions:

| Region       | Starting Block | Offset (bytes) | Size                  | Description                |
|--------------|----------------|----------------|-----------------------|----------------------------|
| Superblock   | 0              | 0              | 4 KB (1 block)        | Global filesystem metadata |
| Block Bitmap | 1              | 4 KB           | 4 KB (1 block)        | Tracks free/used blocks    |
| Inode Table  | 2              | 8 KB           | 32 KB (8 blocks)      | File metadata (256 inodes) |
| Data Blocks  | 10             | 40 KB          | 9.96 MB (2550 blocks) | File contents              |

## Filesystem components

### Superblock

The superblock is the first block in the filesystem and contains critical metadata about the entire filesystem structure. It serves as the “table of contents” for your filesystem.

```
typedef struct {
    int total_blocks;    // Total number of blocks (2560 for 10 MB)
    int block_size;      // Size of each block (4096 bytes)
    int free_blocks;     // Number of available blocks
    int total_inodes;    // Total number of inodes (256)
    int free_inodes;     // Number of available inodes
} superblock;
```

When implementing the superblock:

- During `fs_format()` , initialize all fields to their appropriate values.
- During `fs_mount()` , read the superblock to verify this is a valid filesystem.
- Update `free_blocks` and `free_inodes` when allocating or freeing resources.
- Always write the updated superblock back to disk after changing its values.

## Block bitmap

The block bitmap keeps track of which blocks are free ( 0 ) or in use ( 1 ). Each bit in the bitmap corresponds to one block in the filesystem.

```
// Bitmap is stored as: unsigned char bitmap[MAX_BLOCKS / 8];

// To mark block N as used
bitmap[N/8] |= (1 << (N%8));

// To mark block N as free
bitmap[N/8] &= ~(1 << (N%8));

// To check if block N is used
if (bitmap[N/8] & (1 << (N%8))) {
    // Block is in use
}
```

The bitmap helps you quickly:

- Find free blocks when allocating space for files.
- Mark blocks as free when deleting files.
- Determine if there is enough space available for a write operation.

## Inode table

The inode table contains metadata for each file in the filesystem. Each inode is 128 bytes in size, and your filesystem supports up to 256 inodes (files).

```
typedef struct {
    int used;                // 1 if inode is active, 0 if free
    char name[MAX_FILENAME]; // File name (up to 28 chars)
    int size;                // File size in bytes
    int blocks[MAX_DIRECT_BLOCKS]; // Block pointers (12 max)
} inode;
```

Key operations with inodes:

- When creating a file, find a free inode and initialize it.
- When writing to a file, update the size and block pointers.
- When reading from a file, use block pointers to locate data.
- When deleting a file, mark the inode as free and release its blocks.

**Note:** With 12 direct block pointers and 4 KB blocks, the maximum file size is 48 KB.

## Data blocks

The data blocks region stores the actual contents of files. Each file can use up to 12 blocks (as specified by `MAX_DIRECT_BLOCKS` ), which means the maximum file size is 48 KB.

## API specification

---

### Filesystem operations

**int fs\_format(const char \*disk\_path)**

- **Description:** Creates and initializes a virtual disk file and prepares the filesystem.
- **Parameters:** `disk_path` — path where the virtual disk file will be created.
- **Returns:**
  - 0 on success.
  - -1 on failure (e.g., cannot create file).
- **Actions:**
  - If the file exists, it will be overwritten.
  - Creates a 10 MB file.
  - Initializes superblock, bitmap, and inode table.
  - Sets all blocks as free except those used for metadata.

**int fs\_mount(const char \*disk\_path)**

- **Description:** Loads an existing filesystem from a virtual disk file.
- **Parameters:** `disk_path` — path to the virtual disk file.
- **Returns:**
  - 0 on success.
  - -1 on failure (e.g., file does not exist or invalid filesystem).
- **Actions:**
  - Opens the virtual disk file.
  - Reads and validates the superblock.
  - Loads necessary metadata into memory.

**void fs\_unmount()**

- **Description:** Ensures all pending changes are written to disk and closes the filesystem.
- **Returns:** None.
- **Actions:**
  - Flushes any cached data to disk.
  - Closes the virtual disk file.

### File operations

**int fs\_create(const char \*filename)**

- **Description:** Creates a new empty file in the filesystem.
- **Parameters:** `filename` — null-terminated string (max 28 chars excluding null).
- **Returns:**
  - 0 on success.
  - -1 if file already exists.

- -2 if no free inodes available.
- -3 for other errors.

- **Actions:**

- Checks if the filename already exists.
- Finds a free inode.
- Initializes the inode with the filename and zero size.
- Updates the superblock (decrease `free_inodes` ).

**int fs\_delete(const char \*filename)**

- **Description:** Removes a file and frees its blocks.

- **Parameters:** `filename` — null-terminated string.

- **Returns:**

- 0 on success.
- -1 if file does not exist.
- -2 for other errors.

- **Actions:**

- Finds the file's inode.
- Marks all of the file's blocks as free in the bitmap.
- Marks the inode as free.
- Updates the superblock (increase `free_blocks` and `free_inodes` ).

**int fs\_list(char filenames[][MAX\_FILENAME], int max\_files)**

- **Description:** Lists files in the filesystem.

- **Parameters:**

- `filenames` — pre-allocated 2D array to receive filenames.
- `max_files` — maximum number of filenames to retrieve.

- **Returns:** Number of files found ( 0 to `max_files` ), or -1 on error.

- **Actions:**

- Scans the inode table for used inodes.
- Copies filenames to the provided array.
- Returns the number of files found.

**int fs\_write(const char \*filename, const void \*data, int size)**

- **Description:** Writes data to a file, overwriting any existing content.

- **Parameters:**

- `filename` — target file.
- `data` — pointer to data buffer.
- `size` — number of bytes to write.

- **Returns:**

- 0 on success.
- -1 if file does not exist.
- -2 if out of space (not enough free blocks).
- -3 for other errors.

- **Actions:**

- Finds the file's inode.
- Calculates how many blocks are needed.



- Frees any previously allocated blocks.
- Allocates new blocks as needed.
- Writes data to the allocated blocks.
- Updates the inode (size and block pointers).
- Updates the bitmap and superblock.

**int fs\_read(const char \*filename, void \*buffer, int size)**

- **Description:** Reads file content into a buffer.
- **Parameters:**
  - `filename` — source file.
  - `buffer` — pre-allocated buffer to receive data.
  - `size` — buffer size in bytes.
- **Returns:**
  - Number of bytes read on success.
  - -1 if file does not exist.
  - -3 for other errors.
- **Actions:**
  - Finds the file's inode.
  - Determines how many bytes to read (minimum of file size and buffer size).
  - Reads data from the file's blocks into the buffer.
  - Returns the number of bytes read.

## Implementation requirements

---

### System call usage

Your code must use low-level system calls to access the virtual disk:

```
// Opening the virtual disk
int disk_fd = open(disk_path, O_RDWR | O_CREAT, 0644);

// Reading a block
lseek(disk_fd, block_num * BLOCK_SIZE, SEEK_SET);
read(disk_fd, buffer, BLOCK_SIZE);

// Writing a block
lseek(disk_fd, block_num * BLOCK_SIZE, SEEK_SET);
write(disk_fd, buffer, BLOCK_SIZE);

// Closing the disk
close(disk_fd);
```

Do not use:

- High-level stdio functions ( `fopen` , `fread` , `fwrite` , etc.)
- Memory mapping ( `mmap` )
- Dynamic memory allocation ( `malloc` , `calloc` , etc.)

## Block access pattern

For all file operations, you should follow this general pattern:

1. Find the target file's inode (if the operation requires an existing file).
2. Calculate which blocks are involved in the operation.
3. Read or write those blocks using `lseek` to position and `read` / `write` for I/O.
4. Update metadata (inodes, bitmap, superblock) as needed.
5. Write changes back to disk.

## Header file modification

You **cannot** modify the header file `fs.h`, including the structs.

## Dynamic memory allocation

Dynamic memory allocation (i.e., `malloc`) is **not** allowed and is not needed.

## Simplifying assumptions

You may assume the following:

1. Only a single-threaded process will use your library. Therefore, thread-safety (mutual exclusion) considerations can be omitted.
2. All virtual disk files passed to `fs_mount` are valid filesystems previously created by `fs_format`. You do not need to handle corrupted files, tampered metadata, or any other on-disk integrity errors.

## Implementation steps

---

We recommend implementing the functions in this order:

1. `fs_format` — Start by creating the disk and initializing structures.
2. `fs_mount` and `fs_unmount` — Basic filesystem access.
3. `fs_create` and `fs_list` — Simple file management.
4. `fs_write` and `fs_read` — Data access.
5. `fs_delete` — Resource cleanup.

## Sample helper functions

---

These are some useful helper functions to consider implementing:

```
// Find an inode by filename
int find_inode(const char *filename);

// Find a free inode
int find_free_inode();

// Find a free block
int find_free_block();

// Mark a block as used
void mark_block_used(int block_num);

// Mark a block as free
void mark_block_free(int block_num);

// Read an inode from disk
void read_inode(int inode_num, inode *target);

// Write an inode to disk
void write_inode(int inode_num, const inode *source);
```

## The library header: fs.h

---

```
#ifndef _FS_H
#define _FS_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>

/* ===== */
/*                               Static Constants                               */
/* ===== */

/** Size of each block in bytes. */
#define BLOCK_SIZE 4096

/** Total number of blocks in the 10 MB virtual disk. */
#define MAX_BLOCKS 2560

/** Maximum number of files (inodes). */
#define MAX_FILES 256

/** Maximum filename length (including null terminator). */
#define MAX_FILENAME 29

/** Maximum number of direct block pointers per inode. */
#define MAX_DIRECT_BLOCKS 12

/* ===== */
/*                               On-Disk Data Structures                               */
/* ===== */

/**
 * @brief Superblock structure.
 *
 * Stored at block 0. Contains global filesystem metadata.
 */
typedef struct {
    int total_blocks;    /**< Total number of blocks (2560 for 10 MB). */
    int block_size;      /**< Size of each block (4096 bytes). */
    int free_blocks;     /**< Number of available blocks. */
    int total_inodes;    /**< Total number of inodes (256). */
    int free_inodes;     /**< Number of available inodes. */
} superblock;

/**
 * @brief Inode structure.
```

```

*
* Each inode is 128 bytes. Stores metadata for a single file.
*/
typedef struct {
    int used;                /**< 1 if inode is active, 0 if free. */
    char name[MAX_FILENAME]; /**< File name (up to 28 chars excluding null). */
    int size;                /**< File size in bytes. */
    int blocks[MAX_DIRECT_BLOCKS]; /**< Direct block pointers (12 max). */
} inode;

/* ===== */
/*                               External Interface                               */
/* ===== */

/**
 * @brief Creates and initializes a virtual disk file.
 *
 * If the file already exists, it is overwritten. Creates a 10 MB file and initializes
 * the superblock, bitmap, and inode table. All blocks except metadata blocks are marked
 * free.
 *
 * @param disk_path Path where the virtual disk file will be created.
 * @return 0 on success; -1 on failure.
 */
int fs_format(const char *disk_path);

/**
 * @brief Loads an existing filesystem from a virtual disk file.
 *
 * Opens the virtual disk file, reads and validates the superblock,
 * and loads necessary metadata into memory.
 *
 * @param disk_path Path to the virtual disk file.
 * @return 0 on success; -1 on failure.
 */
int fs_mount(const char *disk_path);

/**
 * @brief Unmounts the filesystem.
 *
 * Flushes any cached data to disk and closes the virtual disk file.
 */
void fs_unmount();

/**
 * @brief Creates a new empty file in the filesystem.
 *
 * @param filename Null-terminated string (max 28 chars excluding null).
 * @return 0 on success; -1 if file already exists; -2 if no free inodes; -3 for other
 * errors.
 */
int fs_create(const char *filename);

```

```

/**
 * @brief Removes a file and frees its blocks.
 *
 * @param filename Null-terminated string.
 * @return 0 on success; -1 if file does not exist; -2 for other errors.
 */
int fs_delete(const char *filename);

/**
 * @brief Lists files in the filesystem.
 *
 * @param filenames Pre-allocated 2D array to receive filenames.
 * @param max_files Maximum number of filenames to retrieve.
 * @return Number of files found (0 to max_files); -1 on error.
 */
int fs_list(char filenames[][MAX_FILENAME], int max_files);

/**
 * @brief Writes data to a file, overwriting any existing content.
 *
 * @param filename Target file.
 * @param data Pointer to data buffer.
 * @param size Number of bytes to write.
 * @return 0 on success; -1 if file does not exist; -2 if out of space; -3 for other
errors.
 */
int fs_write(const char *filename, const void *data, int size);

/**
 * @brief Reads file content into a buffer.
 *
 * @param filename Source file.
 * @param buffer Pre-allocated buffer to receive data.
 * @param size Buffer size in bytes.
 * @return Number of bytes read on success; -1 if file does not exist; -3 for other
errors.
 */
int fs_read(const char *filename, void *buffer, int size);

#endif /* _FS_H */

```

## Common pitfalls

---

- Forgetting to update the superblock after allocating/freeing resources.
- Not writing changes back to disk (especially metadata).
- Buffer overflows with filenames.
- Not handling partial reads/writes correctly.
- Miscalculating block offsets.
- Forgetting to update the bitmap when allocating/freeing blocks.

## Testing your implementation

---

The provided `test.c` includes a basic test.

You should extend this with additional tests:

- Creating multiple files.
- Writing files of different sizes.
- Filling the filesystem to capacity.
- Deleting files and reusing the space.
- Testing error conditions (file not found, disk full).

`test.c` contains a `main` function to test basic functionality. If you do not pass all the provided tests, your submission will get a failing grade.

## Final notes

---

- **Read the instructions carefully:** Ensure that you adhere to all restrictions and requirements.
- **Testing:** Test your code thoroughly with various inputs to ensure correctness.
- **Code quality:** Write clean, well-commented code to help the graders understand your implementation.
- **Error handling:** Pay special attention to error cases and provide appropriate error codes.
- **Grading reminder:** A happy grader is a merciful grader!

---

Good Luck! 🍀